Handover Ledger

Individual Role Blueprint: Product, UI/UX & Authentication / Access Design

Team Member	Panav Khungar - Product & Frontend Systems
Project	Handover Ledger: An Evidence-Linked, Role-Scoped Knowledge Reconstruction Platform for Team Continuity
Teammate	Padminish Bakshi - Backend & AI Systems (claim extraction, ingestion, citation verifier)
Submission	Round 1 Technical Proposal - individual contribution document

1. My Role in One Paragraph

I own everything the user sees and every rule that decides who may see it. On the product side that means the UX workflow, user research, and the continuation-roadmap design. On the engineering side it means the Next.js dashboard, the side-by-side citation viewer, the review and edit console, the role-based redaction renderer, the successor acknowledgement flow, and the authentication and role-permission layer that connects to the database-level Row-Level Security that Padminish's backend relies on. My goal is that a successor can open Handover Ledger on Day 1, understand what is true, what is uncertain and what to do next, without ever seeing something they are not cleared to see.

2. Design Principles

- **Evidence before prose.** No claim is shown without a one-click path to its source span. Citations are first-class UI, not footnotes.
- **Uncertainty is visible.** Verified, Inferred, Conflicting and Missing each have a distinct label, colour and icon. Colour is never the only signal (accessibility).
- **Restricted means absent, not blurred.** Content a role cannot access is removed server-side. The UI shows a neutral placeholder ("Restricted - 3 items hidden for your role") without leaking titles or counts that reveal sensitive details.
- **Action over archive.** The default landing view for a successor is the 30-day plan, not a document dump.
- **Humans stay in control.** Every model output is editable, rejectable and logged before it reaches a successor.

3. Users and Primary Journeys

Persona	Primary goal in the UI	Key screens
Departing Member	Finish a handover in about five minutes without writing documentation from scratch.	Interview inbox, question card, attestation confirm
Project Lead / Reviewer	Decide what is trustworthy and approve a release package.	Claims table, conflict resolver, release scope selector
Incoming Successor	Know what to do first and who owns what, within their clearance.	Continuation plan, evidence viewer, acknowledgement
Org Admin	Control sources, roles and see who viewed or acknowledged what.	Source labelling, role manager, audit log

4. Screen-by-Screen UX Specification

S1. Source Upload & Sensitivity Tagging

On upload each source (repo, CSV/JSON task export, notes, chat export) must be given a sensitivity label (Public-to-team / Restricted / Confidential) and an allowed-roles list before ingestion can start. Upload is blocked until the label is set, so nothing enters the pipeline unclassified. A progress panel shows ingestion state per source.

S2. Claim Ledger (Lead Review Console)

A filterable table of atomic claims: type (decision, owner, blocker, risk, open question), status badge, confidence, source count. Row actions: Approve, Modify, Reject, Add claim. Bulk approve is allowed only for Verified claims. Every change writes a version entry with author and timestamp, shown in a per-claim history drawer.

S3. Side-by-Side Citation Viewer

Split view: claim on the left; the raw source on the right scrolled to and highlighting the exact verbatim span returned by the backend. Multiple citations appear as tabs. Claims whose span failed the backend string check never reach this screen; they appear only in an admin rejected-claims list.

S4. Conflict Resolver

When two sources or the leaver disagree, both claims are shown side by side with their evidence. The lead can choose one, keep both flagged, or write a reconciling claim. The resolution and reasoning are stored in the audit trail. Conflicts are never auto-merged in the UI.

S5. Departing-Member Interview Loop

A card-style flow of 5 to 10 gap-driven questions, each showing why it is being asked (for example, "A cron job is referenced with no runbook"). The leaver answers by text, can skip with a reason, and each answer becomes a claim marked as a direct attestation. A progress bar and Save and resume keep the session short and low-pressure.

S6. Role-Scoped Handover View & 30-Day Plan

The successor sees Day-1, Week-1 and Month-1 sections with checklist items, each linked to supporting claims and an owner. Items depend on role scope, so a Core Dev and a Project Coordinator see different plans from the same ledger. Completed items persist per user.

S7. Acknowledgement Screen

The successor reviews a summary of what was released to them, ticks explicit confirmations (understood open risks, understood ownership transfers), and signs. The system stores a hashed receipt (see section 7).

5. Role & Permission Matrix (Design Deliverable)

Four application roles are proposed for Round 1. Source-level labels decide the ceiling; the role decides what within that ceiling is visible. Matrix below is the working draft and will be validated with project leads in Round 1 research.

Capability	Org Admin	Project Lead	Departing Member	Successor (scoped)
Upload / label sources	Yes	Yes	No	No
View all claims in project	Yes	Yes	Own attestations only	Role-scoped only
Approve / edit / reject claims	Yes	Yes	Own answers only	No
Resolve conflicts	Yes	Yes	No	No
Answer interview questions	No	No	Yes	No
View Restricted sources	Yes	If granted	If granted	If granted
View Confidential sources	Yes	Explicit grant only	No	No

Capability	Org Admin	Project Lead	Departing Member	Successor (scoped)
Create release package / scope	Yes	Yes	No	No
Acknowledge receipt	No	No	No	Yes
Read audit log	Yes	Project only	No	No

6. Authentication & Access Control Design

The stack uses Supabase Auth (with NextAuth as a fallback option) and PostgreSQL Row-Level Security. My part is the identity, session and role design and the frontend enforcement; Padminish's part is the retrieval pipeline that consumes these rules.

6.1 Identity & sessions

- Sign-in via email magic link and Google OAuth (suitable for student clubs and small labs). Short-lived access tokens with refresh rotation.
- Invite-only membership: a user can join a project only through an admin or lead invitation tied to a specific role.
- Role and project membership are stored in database tables, never trusted from client-supplied values or editable JWT claims.

6.2 Authorization layers (defence in depth)

Layer	Mechanism	Owner
1. Database	RLS policies on sources, chunks, claims, releases: a row is readable only if the user's project role satisfies the row's sensitivity label and allowed_roles.	Joint (schema: Padminish; policy rules: Panav)
2. Retrieval	Role filter applied before vector search and before LLM context assembly, so restricted text never enters a prompt for an unauthorized view.	Padminish
3. API	FastAPI dependency verifies the token and resolves role per request; endpoints return only role-filtered payloads.	Padminish
4. Frontend	Server-rendered pages fetch already-filtered data; the redaction renderer shows neutral placeholders and hides restricted actions. UI hiding is a convenience, never the security boundary.	Panav
5. Audit	Every view of a Restricted or Confidential item, every permission change, and every release is logged.	Joint

6.3 Role-based redaction renderer

- The server returns a claim object with a *redacted* flag and no restricted fields; the client renders a placeholder component only.
- Placeholders do not reveal the source name, claim title, or exact count of hidden items beyond a coarse indicator, avoiding inference leaks.
- Search, tooltips, URL parameters and exports go through the same filtered endpoints so there is no side channel.
- Test plan: for each role, a synthetic dataset seeded with known restricted markers; automated tests assert those markers never appear in any rendered page, API response or export for lower roles.

6.4 Successor acknowledgement & audit receipt

- At signing, the system builds a receipt containing the user ID, project, release package ID, role scope, list of released claim IDs with version numbers, and a timestamp.
- The receipt is hashed (SHA-256) and stored append-only; the hash is shown to the user and can be re-verified later to show the released content has not changed.

- Scope of the guarantee: it proves what was released and when, not that the successor read or understood it. The UI wording will say so plainly.
- Releases are immutable: any later edit creates a new version and a new release, leaving the earlier receipt intact.

7. Continuation Roadmap UI Specification

The roadmap is generated from claims (owners, blockers, risks, open questions) by the backend, and I design how it is presented and used.

Horizon	Typical content	UI behaviour
Day 1	Access setup, repo and environment run-through, who to contact, critical blockers	Checklist with owner and linked evidence; items that depend on a restricted resource show as "request access"
Week 1	Orphaned scripts or cron jobs to adopt, open questions to confirm with the lead	Task cards with status and "mark understood" actions; unresolved Conflicting claims surfaced here
Month 1	Planned milestones, risk follow-ups, documentation gaps to close	Timeline view with progress; export of the plan for the successor

8. Accessibility & Usability Commitments

- Keyboard-navigable citation viewer and tables; visible focus states; semantic landmarks and ARIA labels.
- Status badges use text plus icon plus colour; contrast ratio target of WCAG 2.1 AA.
- Responsive layout; the interview loop and acknowledgement flow are fully usable on a phone, since leavers often finish tasks on mobile.
- Plain-language microcopy that explains each badge (for example, "Inferred: the system concluded this; no source states it directly").

9. Round 1 Research Plan & Deliverables

Round 1 outputs from my side are planned as follows. Findings will be reported only from real sessions; any demo data is synthetic.

Deliverable	Description	Status in this submission
UX workflow diagrams	End-to-end flows for the four personas across the five product steps	In blueprint (this document, sections 3-4)
Role-permission matrix	Draft capability matrix and sensitivity-label rules	Drafted (section 5); to be validated
Wireframes	Low/high-fidelity layouts for S1-S7, prioritising S2, S3, S6	Planned for Round 1 prototype
User research	Short interviews with 4 to 6 team leads or club officers and a problem-validation survey (joint)	Planned; no results claimed yet
Roadmap UI spec	Day-1 / Week-1 / Month-1 component specification	Drafted (section 7)

10. Engineering Plan by Round

Round	My engineering scope
Round 2 - Functional MVP	Next.js app shell and auth; project and invite flow; source upload with sensitivity labels (S1); claim ledger and review console (S2); side-by-side citation viewer (S3); single-role export.
Round 3 - Hardened System	Multi-role redaction renderer and RLS policy tests; interview loop UI (S5); conflict resolver (S4); full role-scoped 30-day plan (S6); acknowledgement and hashed receipt (S7); accessibility pass; recorded walkthrough (joint).

11. UX & Access Success Metrics

- **Leak tests:** zero restricted-marker appearances across all role views, APIs and exports in the synthetic test suite.
- **Verifiability:** every claim displayed has a working path to its source span or a leaver attestation.
- **Task success:** in moderated tests with synthetic projects, target that most reviewers can locate a claim's evidence and a successor can state their Day-1 tasks without assistance. Thresholds will be set after initial pilot sessions rather than assumed now.
- **Leaver effort:** interview loop designed around a five-minute completion target; actual time will be measured in tests.

12. Risks & Mitigations (My Area)

Risk	Mitigation
Role matrix too rigid for real teams	Validate with leads; keep roles configurable in schema; start with four roles only.
Placeholders leak information by their presence	Coarse placeholders only; test inference with seeded datasets.
Reviewer fatigue on large claim lists	Default sorting by risk and status; bulk-approve only Verified claims; filters and saved views.
Overstated trust in badges or receipts	Plain-language explanations and honest receipt wording (section 6.4).
Frontend-only enforcement mistakes	Treat UI as convenience; rely on RLS and API filtering as the real boundary.

13. Declaration

Prepared by Panav Khungar as the individual contribution document for the Handover Ledger Round 1 submission, in alignment with the joint team blueprint. All demonstration data, repositories and testimonials will be synthetic or sanitized. AI assistance was used for structural formatting and drafting support; the design decisions, role matrix and access model are the author's own work and have been reviewed by the author.